Poker Hand Classification With Supervised Learning

By: Kavin Ezhilan

Course: ITCS 3156

Date: 5/2/2026

Introduction

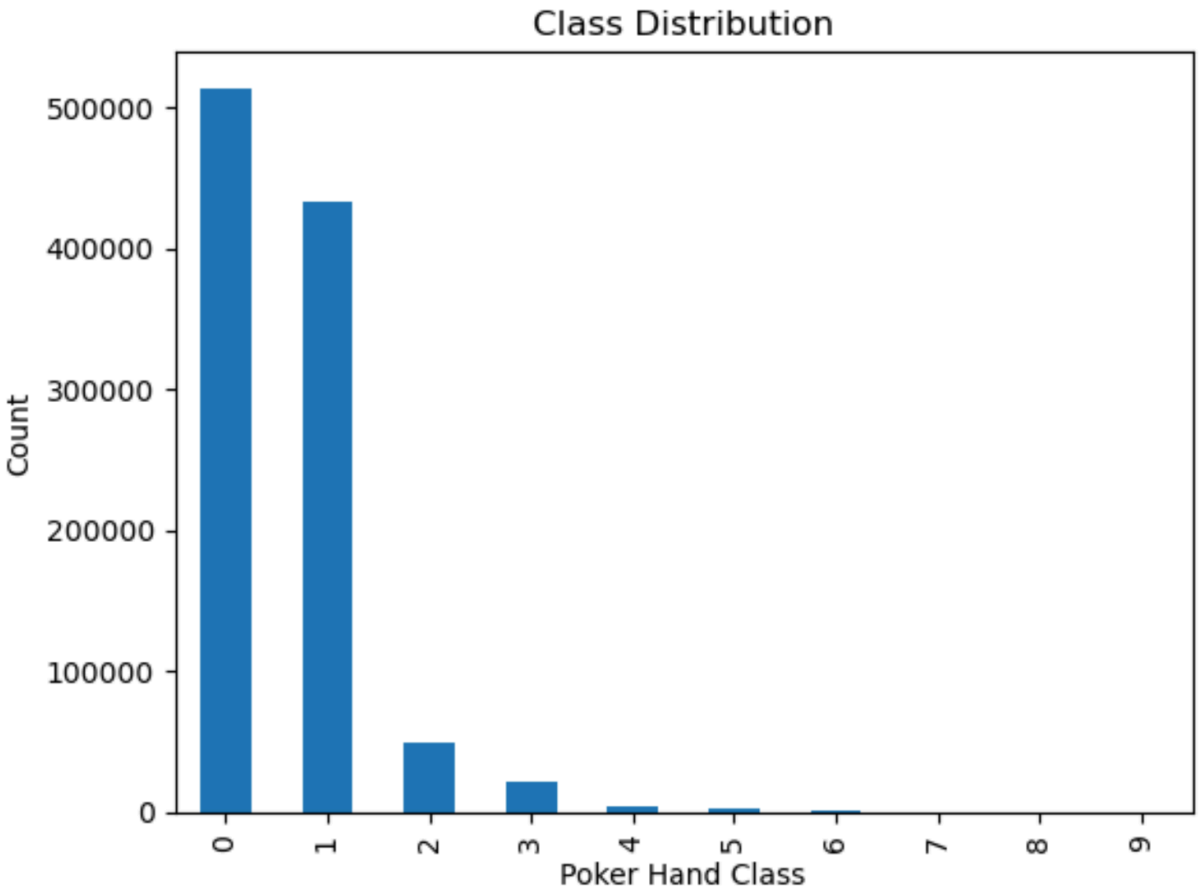
Github Repo: https://github.com/kezhilan/ITCS3156#

Problem Statement: This project was designed to classify poker hands into ten distinct categories ranging from 0 - 9 based on the combination of five cards. Each card is denoted by a suit and number: 1,1 for Ace of Hearts.

Motivations and Challenges: Identifying poker hands is considered a classification problem as the algorithm would analyze the data to determine which of the ten discrete classes the instance belongs to. However unlike standard classification problems a poker hand is composed of 5 cards and the algorithm must determine which class an instance belongs to based on the combination not the individual card. Another challenge was the dataset itself. The stronger poker hands are much less likely to exist than the weaker ones, as such the dataset was very imbalanced with the majority of the instances belonging to class 0 and class 1 with some of the rarer classes only having a handful of instances in the large dataset which could make classification difficult.

Approach: To address the issue multiple classification algorithms are chosen. KNN, Naive Bayes, and Logistic Regression were all chosen to compare. A distance based classification in KNN, a probabilistic classification in Naive Bayes and a linear model in regression were chosen to see which would be the best at identifying classes correctly in a multivariable and combinational dataset.

Data: The dataset came from the UCL machine learning repository called the Poker Hand Dataset. The dataset contains over 1,000,000 instances each denoting a 5 card poker hand with 10 input features and 10 classes. As I have already mentioned the data is very imbalanced; the instances that dominate the dataset are class 0 and class 1 (High card and One pair respectively). Rare classes like 8 and 9 (the Straight Flush, and Royal Flush respectively) only have a handful of instances and are extremely rare. The bar chart below shows how skewed the data is.



Setup

Data Preprocessing: To preprocess the data there were a few problems I faced. The first is that the dataset is rather large and it would take too long for KNN to calculate the distance between each datapoint. And the dataset is very skewed so as a solution I combined features and labels and made a small sample dataset that takes up to 500 instances from each class and all instances if a class has fewer than 500. I then separated the features and target into smaller datasets. This both reduced class imbalance and made it computationally efficient. The code below is a small segment of how I accomplished this.

```
df = X.copy()
df['CLASS'] = y
balanced = df.groupby('CLASS', group_keys=False).apply(
        lambda x: x.sample(n=min(len(x), 500), random_state=42)
)
X_small = balanced.drop('CLASS', axis=1)
y_small = balanced['CLASS']
```

The data is then split into a training and testing set (80/20) split . There is also feature scaling applied to KNN and Logistic regression since the models depend on numerical distances and weights.

Methods:

KNN: K Nearest Neighbors is an algorithm that computes the distance between instances to determine what class an instance belongs to. It assigns class based on majority voting depending on the set K value. For my purposes I used the euclidean distance measurement with a K value of 5 due to the large number of classes. The model does not assume anything about the distribution and is simple but would be too computationally expensive to run on the entire dataset, hence why I had to make a sample and the class imbalance will make classification for the rarer classes less accurate. However the most effective way to determine a hand will be to look for similarities in the hand construction between other hands of the class.

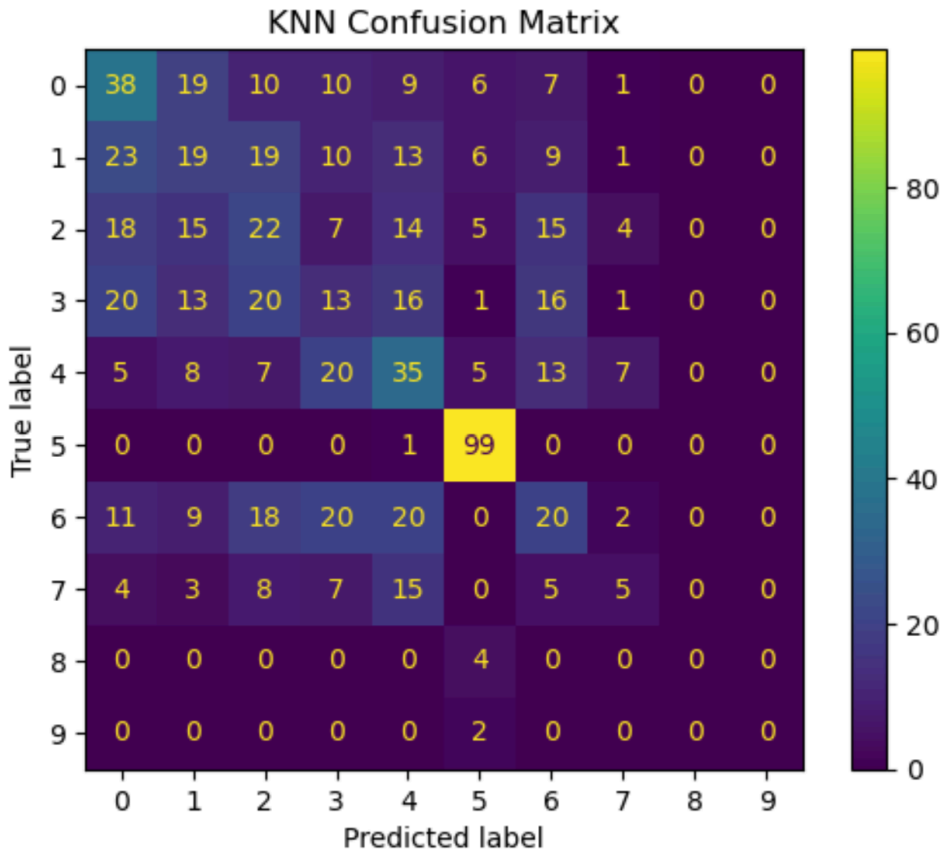
Naive Bayes: Naive Bayes is a probabilistic model that is based on Bayes Theorem. It predicts using the probability of each feature. The specific type I used is Gaussian Naive Bayes that did not require feature scaling. It should be much faster and efficient than KNN and should work well with high dimensional data. However the main issue is that it assumes independence in probabilities which poker hands do not follow as cards/hands and probabilities are heavily determined by what is dealt or in the deck.

Logistic Regression: Logistic Regression is a linear classification model that calculates probability with a sigmoid function. It is also faster than KNN and should work well for any dataset that is linearly separable. It can classify problems with multiple labels but that is only if decision boundaries are clear and linear in nature and are sensitive to scaling.

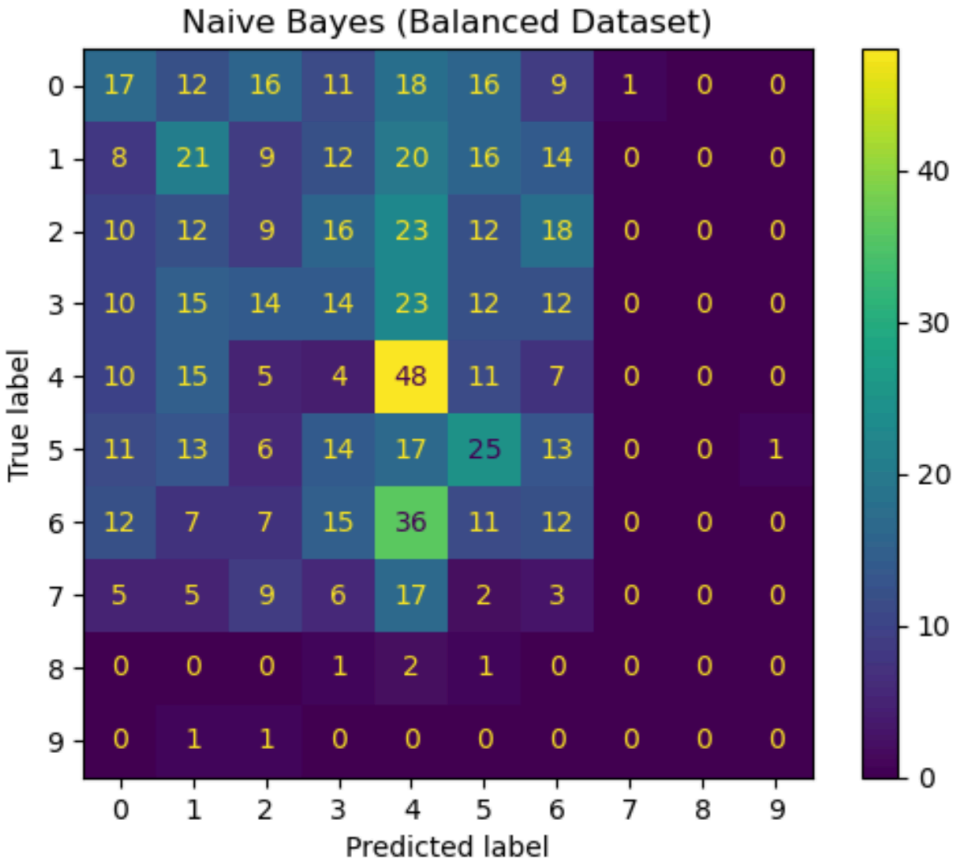
Analysis

Results:

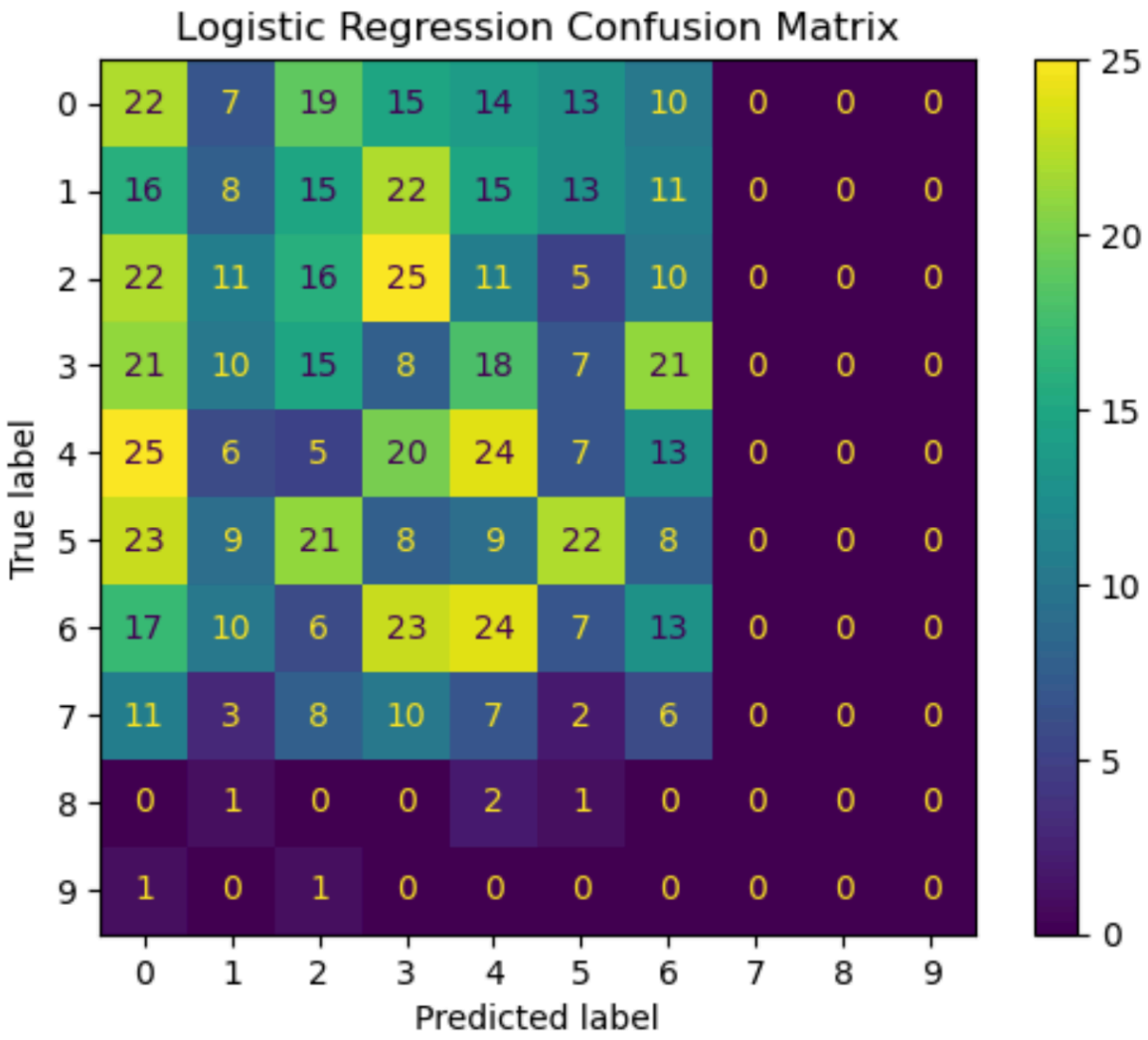
KNN: KNN had the highest overall accuracy score among all the models at 0.33 at k = 5. When the value of k increased the accuracy also increased. K values of 3, 5, 7, and 9 were tested with the larger k values having higher accuracy. This means that comparisons to more neighbors smoothed the decisions and made them more accurate. It also showed remarkable accuracy on certain hands like class 5 or the flush where it had a .99 recall rate. This is because a flush has a very distinct pattern that makes it easy to detect by distance (since all cards are of the same suit). But for the rare classes it performed very poorly most likely because of the low number of instances of said classes in both the training and testing sets due to their rarity with classes 8 and 9 having a 0.00 recall rate. Below is a graph of k = 5 KNN confusion matrix



Naive Bayes: Naive Bayes had an overall accuracy score of 0.19 making it the second best model in this study. The performance was inconsistent and low across the board with the only outlier being class 4 the straight which received a 0.48 recall value most likely due to the sequential nature of the hand. Like KNN the model completely failed at identifying the rare cases especially classes 8 and 9. The reason for the overall poor performance is most likely the computation of Naive Bayes itself as it assumes features are independent of each other but in the case of poker it is different. There is dependence among cards and suits and Naive Bayes was not able to grasp that at all. Overall while being more efficient it does not work on complex datasets like a poker dataset despite its probabilistic approach. Below is the confusion matrix for Naive Bayes.



Logistic Regression: Logistic Regression had the lowest accuracy score out of all the models with a score of 0.15. It did not classify a single class well and completely failed classes 7 and higher. Due to the model assuming a linear decision boundary between classes (which poker hands do not have) it failed across the board. Logical regression is the weakest out of the 3 as it struggles with non linear boundaries and features that interact and depend on each other like a poker hand. Below is the confusion matrix for logistic regression.



Conclusion

This project tackled the issue of poker hand classification using 3 supervised machine learning models. The models chosen were K Nearest Neighbors, Naive Bayes, and Logistic Regression. The goal was to determine how the different models would perform on a combinational dataset with multiple features. The trails showed the strengths and weaknesses of of each model relative to the problem at hand

KNN was the model that received the highest accuracy score regardless of the k value. K values from 3 - 9 were tested showing that an increase in K increased overall accuracy. This model performed better because it was able to determine similarity between classes and instances without assuming any underlying information or probabilities. For example it was very successful in determining flushes because using a distance based approach was highly effective in that case as one of the features would yield a distance of 0 making classification easy. Naive Bayes was the next highest but it struggled overall in most cases due to the features not being independent which Naive Bayes assumes. It only performed better on straights due to the sequential nature of the class. Logistic Regression was by far the worst as it did not perform well in any class. Due to the non linear decision boundaries and interdependence among features Logistic Regression struggled and was basically ineffective in classifying instances correctly.

The most important takeaway was the effect of class imbalance. Due to higher classes having so much less instances for training and testing none of the models were able to predict or analyze any pattern from them leading to utter failure on those rare classes. Even when I tried to sample the dataset and make sure each class was as well represented as I could, the problem persisted and the models were still ineffective on rare classes. Also I learned the importance of sampling as large datasets may take too long to run, this is especially true for KNN as it calculates distance to every instance in the dataset making running it with anything other than a sample computationally costly. Also it showed that simpler models may not always be better and it is imperative to understand the data in the dataset and pick which model fits it better, this is proven by how poorly Logistic models compare to KNN. While none of the models had high accuracy scores it showed how these models behaved in a complex problem and in this specific case distance based measuring was more effective as it could better predict suits and patterns over the other 2.

Acknowledgements

I, Kavin Ezhilan, acknowledge the use of AI in completing this assignment and would like to provide a brief explanation of how I utilized AI, specifically ChatGPT, as a tool to support my work. I used chatgpt mainly to help me create a sample of my dataset and make it representative of the total dataset since the original was too big to run, especially with KNN. I also used it to debug any issues I had with imports and downloads as I had to import the dataset. It also suggested the MinMax scaler I used for preprocessing. I also used it to format references in MLA format.

References

Cattral, Robert and Franz Oppacher. "Poker Hand." UCI Machine Learning Repository, 2002, https://doi.org/10.24432/C5KW38.

NumPy Developers. *NumPy Documentation*. NumPy, https://numpy.org. Accessed 2 May 2026.

Python Software Foundation. *Python Documentation*. https://docs.python.org. Accessed 2 May 2026.

pandas Development Team. *pandas Documentation*. https://pandas.pydata.org. Accessed 2 May 2026.

Waskom, Michael. *Seaborn: Statistical Data Visualization*. https://seaborn.pydata.org. Accessed 2 May 2026.